

Winning Space Race with Data Science

Trần Quang Huy

July 14, 2026

https://github.com/huybedev/ibm_data_science



AGENDA

Outline

01 Executive Summary

02 Introduction

03 Methodology

04 Results

05 Conclusion

How the analysis was built

Methodology

- Collected data via the SpaceX API and web scraping
- Wrangled records and one-hot encoded categorical features
- Explored patterns with SQL and data visualization
- Built interactive analytics with Folium and Plotly Dash
- Trained, tuned and evaluated classification models

Results

- Success rate rises with flight number and with orbit type
- KSC LC-39A shows the strongest track record of any site
- The decision tree classifier best predicts landing outcome

INTRODUCTION

Reusable boosters change the economics of launch

SpaceX advertises Falcon 9 launches at \$62 million, well below competitors' \$165 million and up — largely because the first stage can be reused. If we can predict whether that stage will land successfully, we can estimate the cost of a launch, information any competitor would need to bid against SpaceX.

Key questions

- Which factors determine a successful first-stage landing?
- How do those factors interact to affect the success rate?
- What operating conditions are required for a reliable landing program?

Section 1

Methodology

METHODOLOGY

Six steps from raw data to a working model

01

Collect data via API & web scraping

02

Wrangle data & engineer landing labels

03

Explore patterns with visualization & SQL

04

Map launch sites interactively with Folium

05

Build a live dashboard with Plotly Dash

06

Build, tune & evaluate classification models

Data Collection – SpaceX API

- GET requests to the SpaceX REST API returned launch records as JSON
- Parsed with `.json()` and flattened into a DataFrame via `json_normalize()`
- Cleaned missing values and standardized fields for modeling

```
1. Get request for rocket launch data using API

In [6]: spacex_url="https://api.spacexdata.com/v4/launches/past"

In [7]: response = requests.get(spacex_url)

2. Use json_normalize method to convert json result to dataframe

In [12]: # Use json_normalize method to convert the json result into a dataframe
         # decode response content as json
         static_json_df = res.json()

In [13]: # apply json_normalize
         data = pd.json_normalize(static_json_df)

3. We then performed data cleaning and filling in the missing values

In [30]: rows = data_falcon9['PayloadMass'].values.tolist()[0]

         df_rows = pd.DataFrame(rows)
         df_rows = df_rows.replace(np.nan, PayloadMass)

         data_falcon9['PayloadMass'][0] = df_rows.values
         data_falcon9
```

Data Collection – Web Scraping

- Scraped Falcon 9 launch records from Wikipedia with BeautifulSoup
- Parsed the HTML launch table and converted it into a DataFrame
- Combined with API data for a complete launch history

```
1. Apply HTTP Get method to request the Falcon 9 rocket launch page

In [4]: static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027016923"

In [5]: # use requests.get() method with the provided static_url
# assign the response to a object
html_data = requests.get(static_url)
html_data.status_code

Out[5]: 200

2. Create a BeautifulSoup object from the HTML response

In [6]: # Use BeautifulSoup() to create a BeautifulSoup object from a response text content
soup = BeautifulSoup(html_data.text, 'html.parser')

Print the page title to verify if the BeautifulSoup object was created properly

In [7]: # Use soup.title attribute
soup.title

Out[7]: <title>List of Falcon 9 and Falcon Heavy launches - Wikipedia</title>

3. Extract all column names from the HTML table header

In [38]: column_names = []

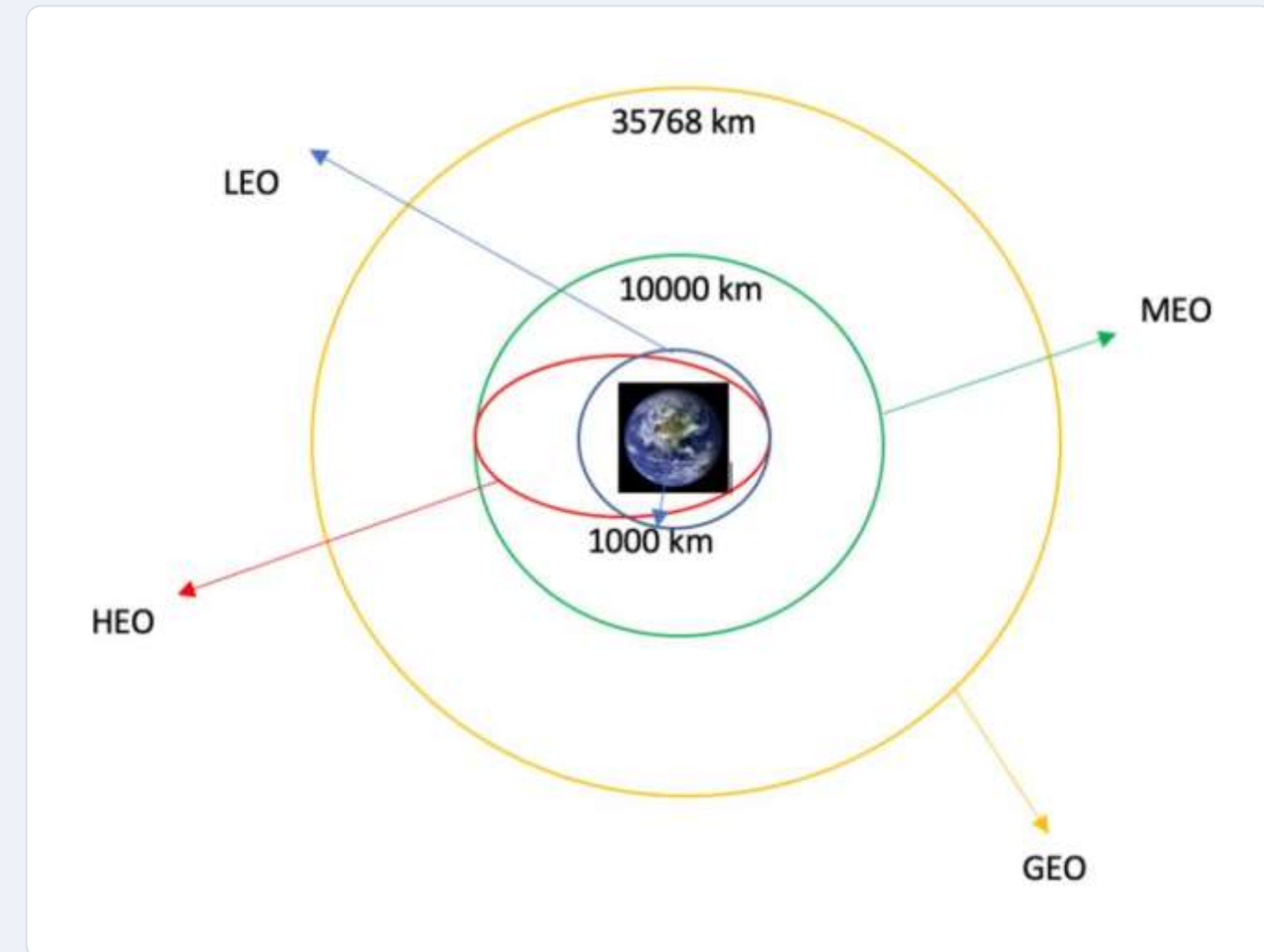
# Apply find_all() function with "th" element on first_launch_table
# Iterate each th element and apply the provided extract_column_from_header() to get a column name
# Append the non-empty column name (if name is not None and len(name) > 0) into a list called column_names

element = soup.find_all('th')
for row in range(len(element)):
    try:
        name = extract_column_from_header(element[row])
        if (name is not None and len(name) > 0):
            column_names.append(name)
    except:
        pass

4. Create a dataframe by parsing the launch HTML tables
5. Export data to csv
```

Data Wrangling

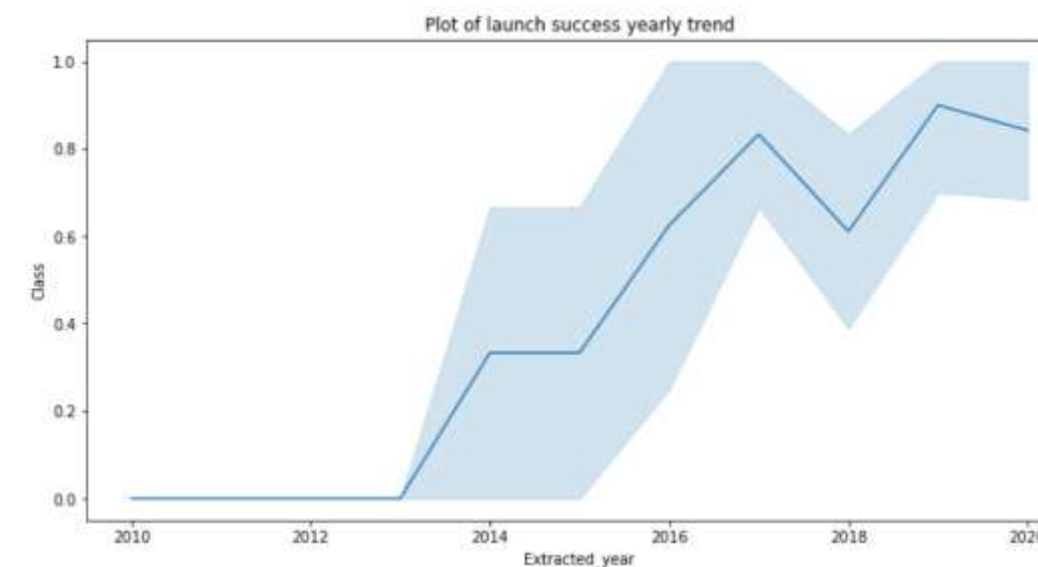
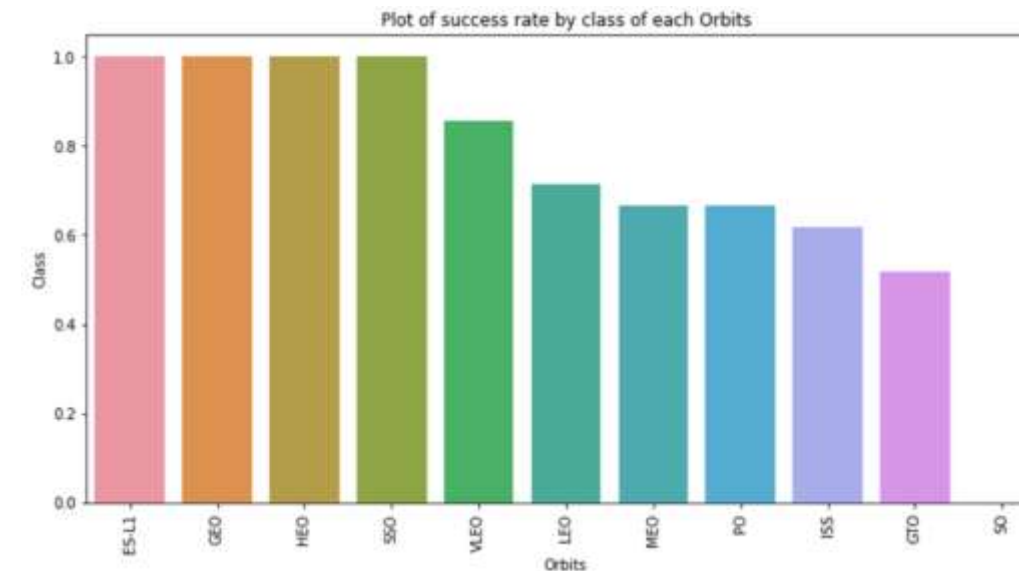
- Counted launches per site and occurrences of each orbit type
- Labeled landing outcomes — success vs. failure — from the outcome column
- Exported the cleaned dataset for exploratory analysis



Reference: orbit classes used to label the dataset

EDA with Data Visualization

- Compared flight number and payload against launch site
- Examined success rate by orbit type and by flight number
- Tracked the yearly trend in launch success



EDA with SQL

- Loaded the SpaceX dataset into PostgreSQL directly from the notebook
- Queried unique launch sites, and total & average payload by booster version
- Filtered landing outcomes by date, launch site and payload range

Build an Interactive Map with Folium

- Marked every launch site with markers, circles and lines for success or failure
- Color-labeled outcomes by marker cluster to spot high-success sites at a glance
- Calculated distance from each site to coastlines, highways and cities

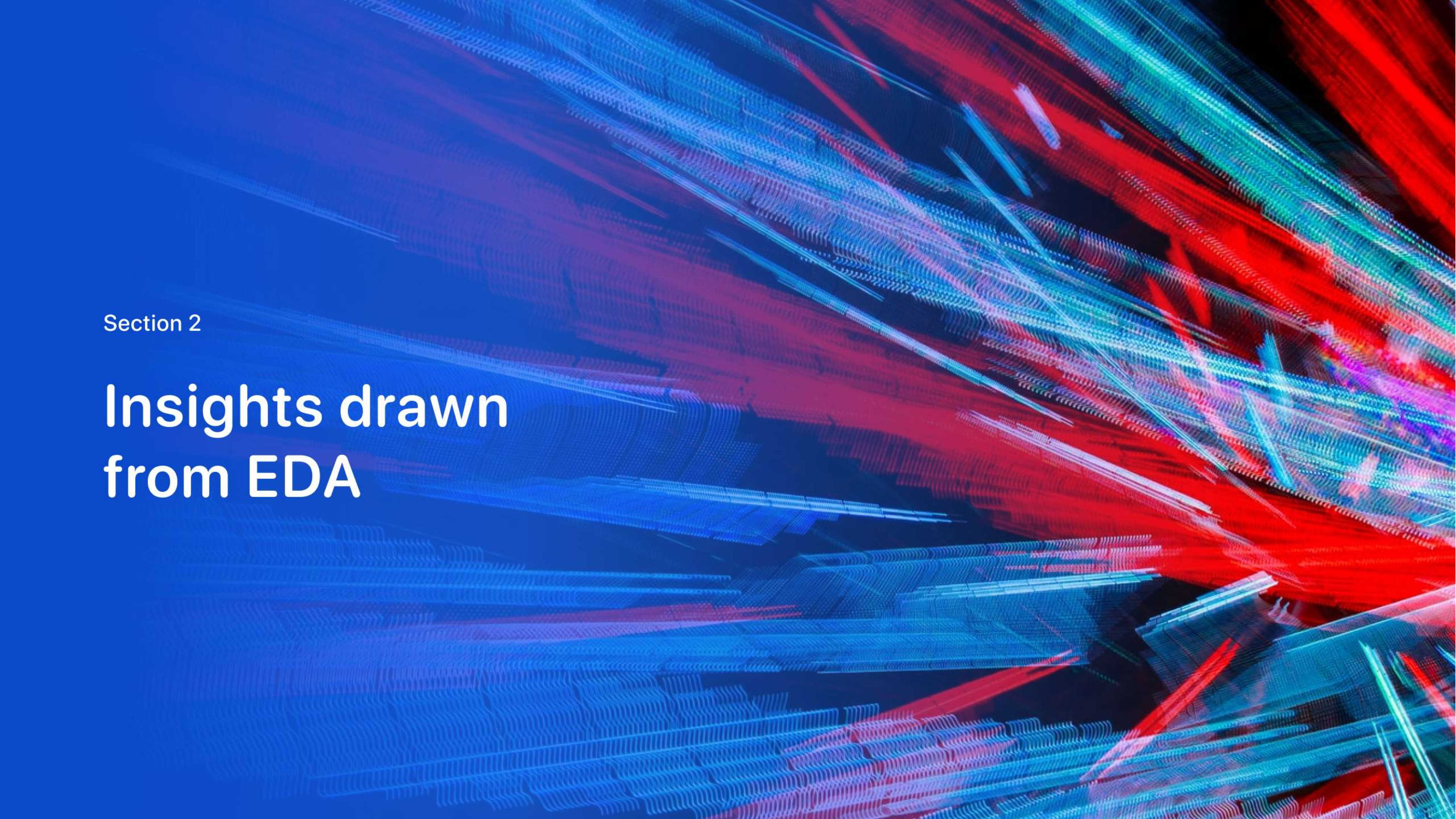
Build a Dashboard with Plotly Dash

- Built an interactive dashboard summarizing launch outcomes
- Pie charts show total launches and success share by site
- Scatter plot relates payload mass to outcome across booster versions

• https://github.com/huybedev/ibm_data_science/blob/main/spacex_dash_app.py

Predictive Analysis (Classification)

- Standardized the features and split the data into training and test sets
- Tuned Logistic Regression, SVM, KNN and Decision Tree models with GridSearchCV
- Selected the best model using classification accuracy on the test set

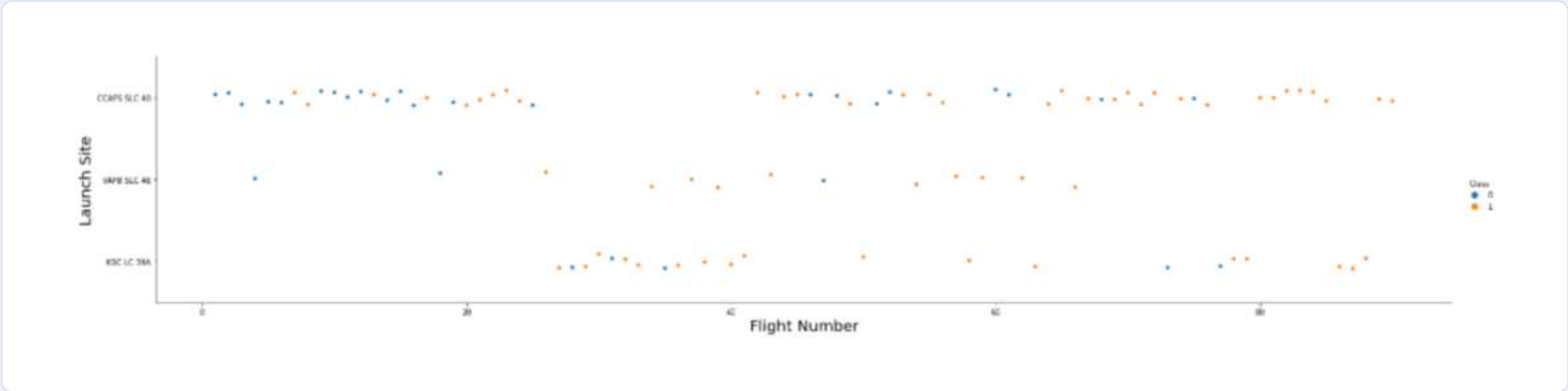
The background of the slide is an abstract composition of numerous thin, overlapping lines and streaks in shades of blue, red, and teal. These lines are oriented diagonally, creating a sense of motion and depth. The overall effect is a complex, layered texture that resembles a digital or data-driven environment.

Section 2

Insights drawn from EDA

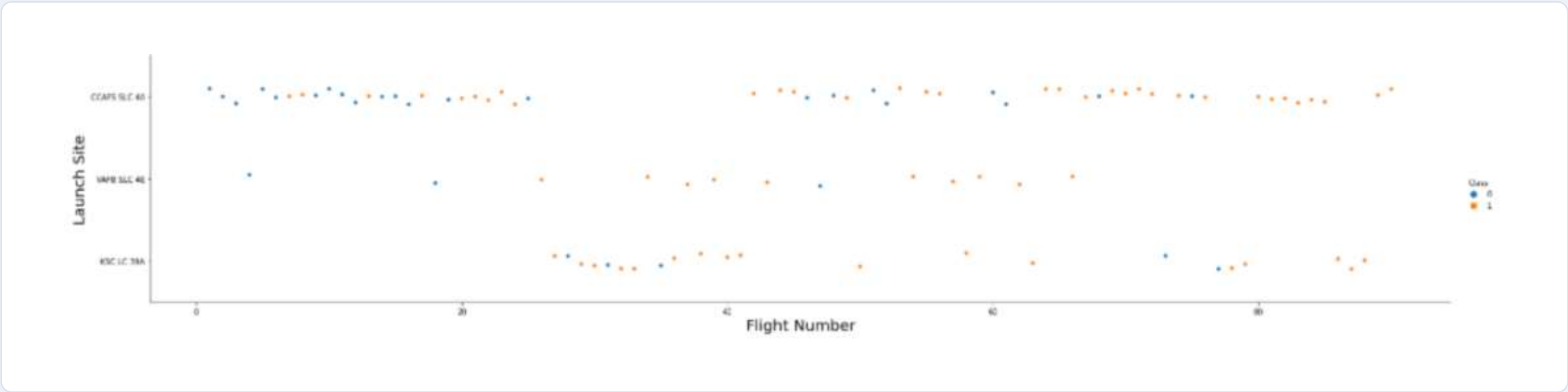
Flight Number vs. Launch Site

Success rates climb as each site accumulates more flights — the trend is clearest at CCAFS SLC-40.



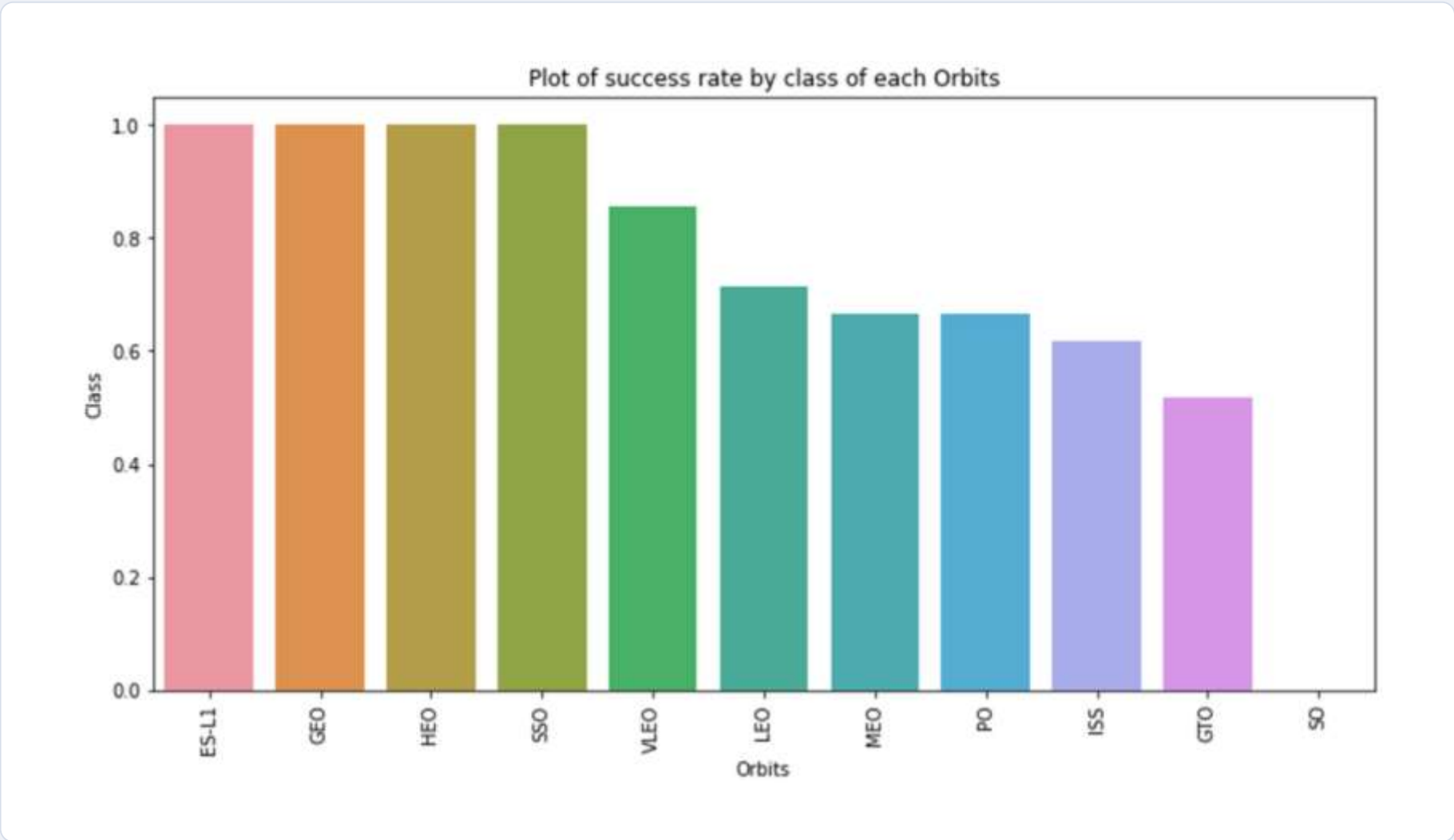
Payload vs. Launch Site

Heavier payloads are concentrated at CCAFS SLC-40 and KSC LC-39A, the two most active sites.



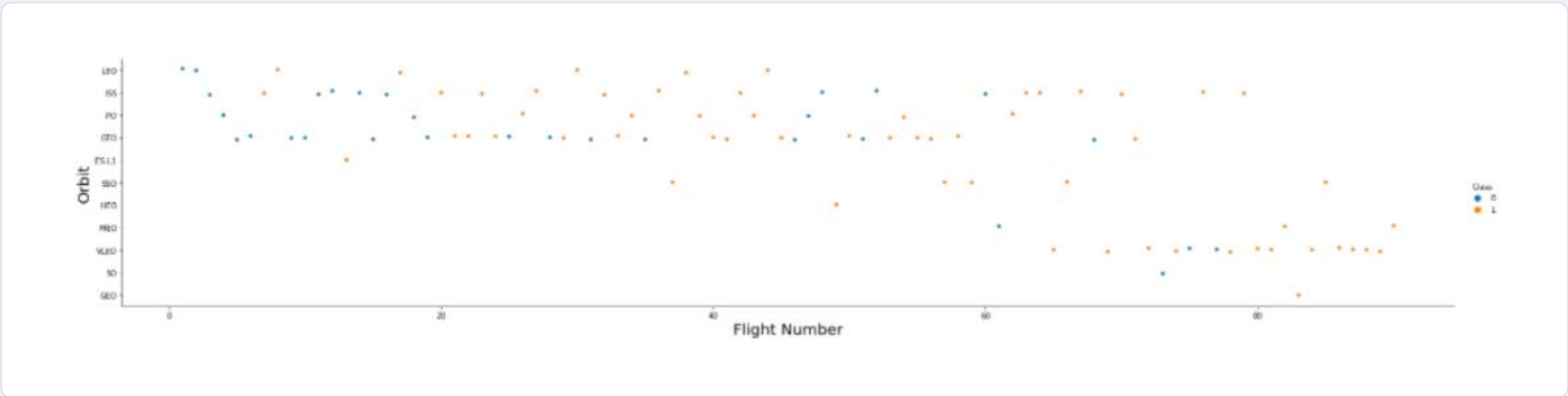
Success Rate vs. Orbit Type

ES-L1, GEO, HEO, SSO and VLEO orbits show the highest landing success rates.



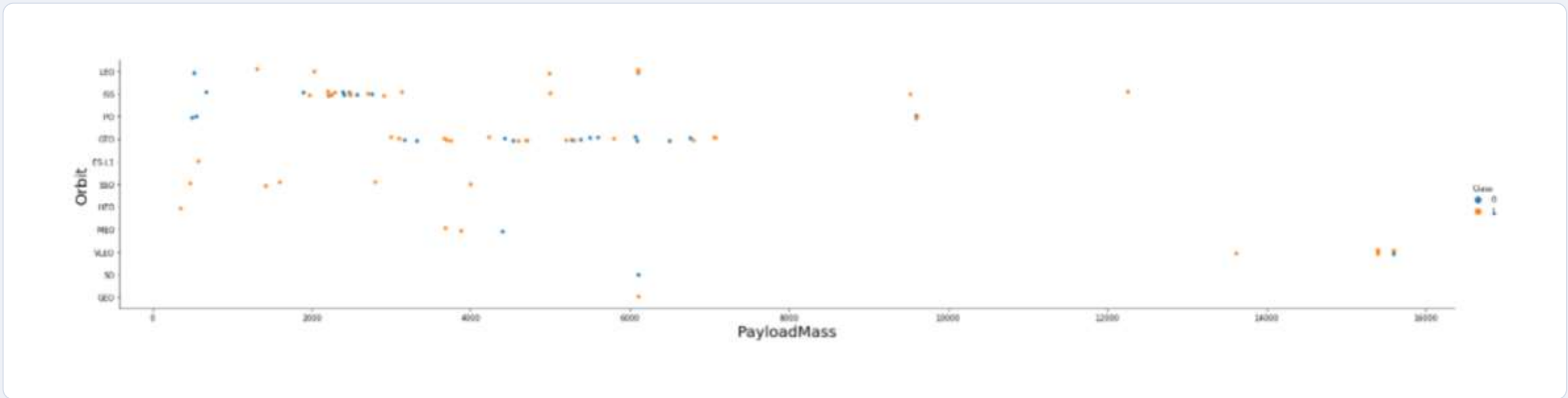
Flight Number vs. Orbit Type

In LEO, success tracks closely with flight number; in GTO, the two show no clear relationship.



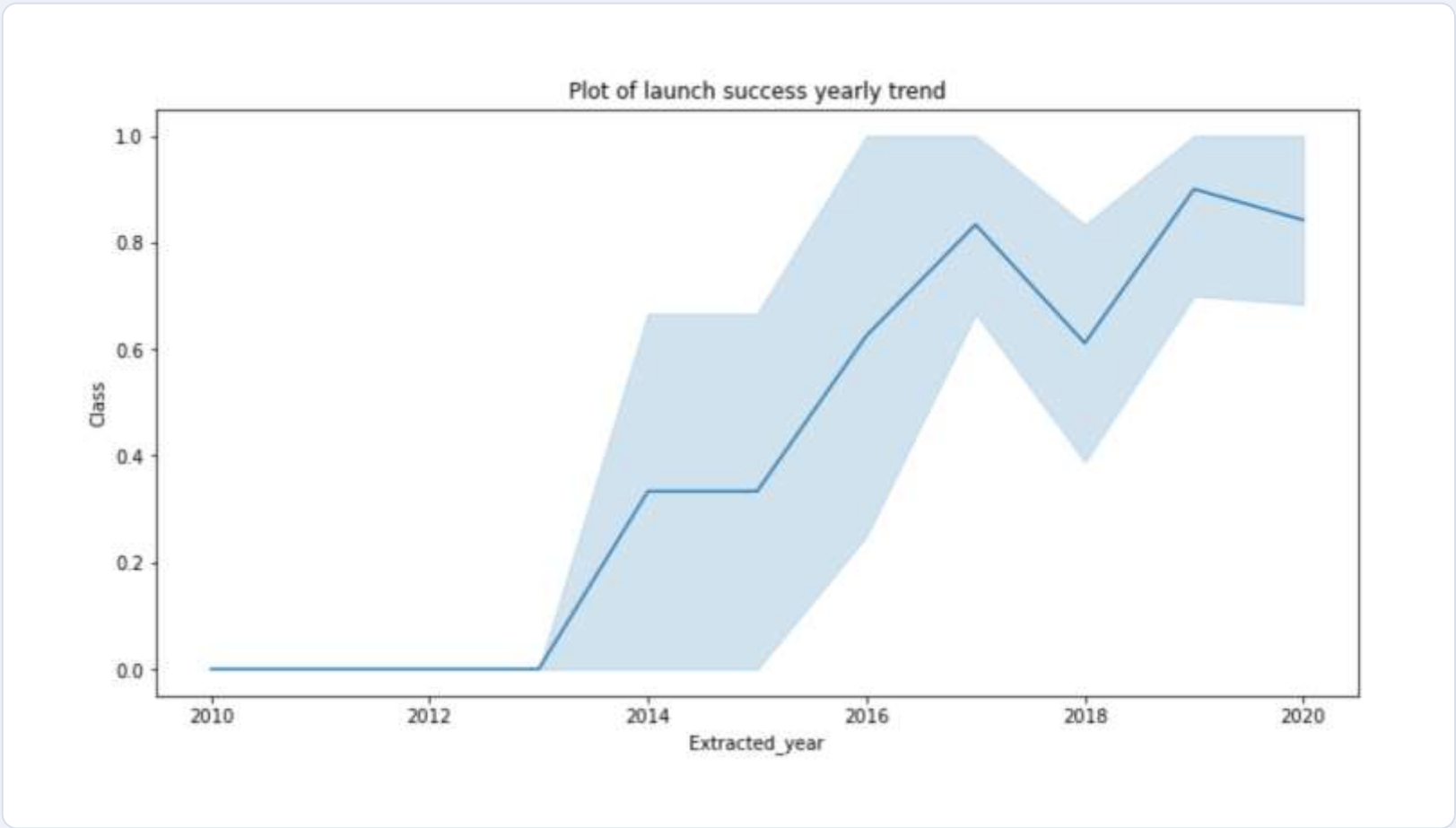
Payload vs. Orbit Type

Heavy payloads land most successfully in PO, LEO and ISS orbits.



Launch Success Yearly Trend

Success rate rose steadily from 2013 through 2020 as the program matured.



All Launch Site Names

```
SELECT DISTINCT LaunchSite FROM SpaceX
```

Four unique launch sites appear across the dataset.

Display the names of the unique launch sites in the space mission

```
In [10]: task_1 = '''
          SELECT DISTINCT LaunchSite
          FROM SpaceX
          '''
          create_pandas_df(task_1, database=conn)
```

```
Out[10]:
```

	launchsite
0	KSC LC-39A
1	CCAFS LC-40
2	CCAFS SLC-40
3	VAFB SLC-4E

Launch Sites Beginning With ‘CCA’

```
SELECT * FROM SpaceX WHERE LaunchSite LIKE 'CCA%' LIMIT 5
```

A LIKE filter surfaces the first five CCAFS records.

Display 5 records where launch sites begin with the string 'CCA'

In [11]:

```
task_2 = '''
SELECT *
FROM SpaceX
WHERE LaunchSite LIKE 'CCA%'
LIMIT 5
'''

create_pandas_df(task_2, database=conn)
```

Out[11]:

	date	time	boosterversion	launchsite	payload	payloadmasskg	orbit	customer	missionoutcome	landingoutcome
0	2010-04-06	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
1	2010-08-12	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of...	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2	2012-05-22	07:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
3	2012-08-10	00:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
4	2013-01-03	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass — NASA

```
SELECT SUM(PayloadMassKG) FROM SpaceX WHERE  
Customer LIKE 'NASA (CRS)'
```

Boosters launched for NASA carried **45,596 kg** in total.

Display the total payload mass carried by boosters launched by NASA (CRS)

```
In [12]: task_3 = '''  
        SELECT SUM(PayloadMassKG) AS Total_PayloadMass  
        FROM SpaceX  
        WHERE Customer LIKE 'NASA (CRS)'  
        '''  
  
        create_pandas_df(task_3, database=conn)
```

```
Out[12]:
```

	total_payloadmass
0	45596

Average Payload Mass — F9 v1.1

```
SELECT AVG(PayloadMassKG) FROM SpaceX WHERE  
BoosterVersion='F9 v1.1'
```

The F9 v1.1 booster averaged **2,928.4 kg** per mission.

Display average payload mass carried by booster version F9 v1.1

```
In [13]: task_4 = '''  
        SELECT AVG(PayloadMassKG) AS Avg_PayloadMass  
        FROM SpaceX  
        WHERE BoosterVersion = 'F9 v1.1'  
        '''  
  
        create_pandas_df(task_4, database=conn)
```

```
Out[13]:
```

	avg_payloadmass
0	2928.4

First Successful Ground-Pad Landing

```
SELECT MIN(Date) FROM SpaceX WHERE  
LandingOutcome LIKE 'Success (ground pad)'
```

The first successful ground-pad landing occurred on
22 December 2015.

```
In [14]: task_5 = '''  
         SELECT MIN(Date) AS FirstSuccessfull_landing_date  
         FROM SpaceX  
         WHERE LandingOutcome LIKE 'Success (ground pad)'  
         '''  
         create_pandas_df(task_5, database=conn)
```

```
Out[14]:
```

	firstsuccessfull_landing_date
0	2015-12-22

Drone-Ship Landings, 4,000–6,000 kg

```
SELECT * FROM SpaceX WHERE LandingOutcome=
'Success (drone ship)' AND PayloadMassKG BETWEEN
4000 AND 6000
```

Four boosters meet both the drone-ship and payload-range criteria.

```
In [15]: task_6 = '''
        SELECT BoosterVersion
        FROM SpaceX
        WHERE LandingOutcome = 'Success (drone ship)'
              AND PayloadMassKG > 4000
              AND PayloadMassKG < 6000
        ...
        create_pandas_df(task_6, database=conn)
```

```
Out[15]:
```

	boosterversion
0	F9 FT B1022
1	F9 FT B1026
2	F9 FT B1021.2
3	F9 FT B1031.2

Successful vs. Failed Mission Outcomes

COUNT(MissionOutcome) WHERE MissionOutcome LIKE 'Success%' / 'Failure%'

100 missions succeeded against **1** recorded failure.

List the total number of successful and failure mission outcomes

```
In [16]: task_7a = '''
          SELECT COUNT(MissionOutcome) AS SuccessOutcome
          FROM SpaceX
          WHERE MissionOutcome LIKE 'Success%'
          '''

          task_7b = '''
          SELECT COUNT(MissionOutcome) AS FailureOutcome
          FROM SpaceX
          WHERE MissionOutcome LIKE 'Failure%'
          '''

          print('The total number of successful mission outcome is:')
          display(create_pandas_df(task_7a, database=conn))
          print()
          print('The total number of failed mission outcome is:')
          create_pandas_df(task_7b, database=conn)
```

The total number of successful mission outcome is:

	successoutcome
0	100

The total number of failed mission outcome is:

```
Out[16]:
```

	failureoutcome
0	1

Booster With Maximum Payload

```
SELECT BoosterVersion FROM SpaceX WHERE
PayloadMassKG = ( SELECT MAX(PayloadMassKG)
FROM SpaceX)
```

Several B5 boosters share the record for heaviest payload carried.

List the names of the booster_versions which have carried the maximum payload mass. Use a subquery

```
In [17]: task_8 = '''
        SELECT BoosterVersion, PayloadMassKG
        FROM SpaceX
        WHERE PayloadMassKG = (
            SELECT MAX(PayloadMassKG)
            FROM SpaceX
        )
        ORDER BY BoosterVersion
        '''
        create_pandas_df(task_8, database=conn)
```

Out[17]:

	boosterversion	payloadmasskg
0	F9 B5 B1048.4	15600
1	F9 B5 B1048.5	15600
2	F9 B5 B1049.4	15600
3	F9 B5 B1049.5	15600
4	F9 B5 B1049.7	15600
5	F9 B5 B1051.3	15600
6	F9 B5 B1051.4	15600
7	F9 B5 B1051.6	15600
8	F9 B5 B1056.4	15600
9	F9 B5 B1058.3	15600
10	F9 B5 B1060.2	15600
11	F9 B5 B1060.3	15600

2015 Launch Records

WHERE LandingOutcome LIKE 'Failure (drone ship)' AND
Date BETWEEN '2015-01-01' AND '2015-12-31'

Two drone-ship landing failures were recorded in
2015, both at CCAFS LC-40.

List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015

In [18]: task_9 = ...
SELECT BoosterVersion, LaunchSite, LandingOutcome
FROM SpaceX
WHERE LandingOutcome LIKE 'Failure (drone ship)'
AND Date BETWEEN '2015-01-01' AND '2015-12-31'
...
create_pandas_df(task_9, database=conn)

Out[18]:

	boosterversion	launchsite	landingoutcome
0	F9 v1.1 B1012	CCAFS LC-40	Failure (drone ship)
1	F9 v1.1 B1015	CCAFS LC-40	Failure (drone ship)

Ranked Landing Outcomes, 2010–2017

```
SELECT LandingOutcome, COUNT(LandingOutcome)
GROUP BY LandingOutcome ORDER BY COUNT(...)
DESC
```

‘No attempt’ and ‘Success (drone ship)’ were the two most common outcomes in this window.

Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad))

```
In [19]: task_10 = '''
        SELECT LandingOutcome, COUNT(LandingOutcome)
        FROM SpaceX
        WHERE DATE BETWEEN '2010-06-04' AND '2017-03-20'
        GROUP BY LandingOutcome
        ORDER BY COUNT(LandingOutcome) DESC
        '''
        create_pandas_df(task_10, database=conn)
```

```
Out[19]:
```

	landingoutcome	count
0	No attempt	10
1	Success (drone ship)	6
2	Failure (drone ship)	5
3	Success (ground pad)	5
4	Controlled (ocean)	3
5	Uncontrolled (ocean)	2
6	Precluded (drone ship)	1
7	Failure (parachute)	1

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The background is a deep blue space with stars. The Earth's surface is dark blue, with bright yellow and orange lights from cities and towns. The lights are concentrated in the lower right quadrant of the image, forming a dense network of glowing points and lines.

Section 3

Launch Sites Proximities Analysis

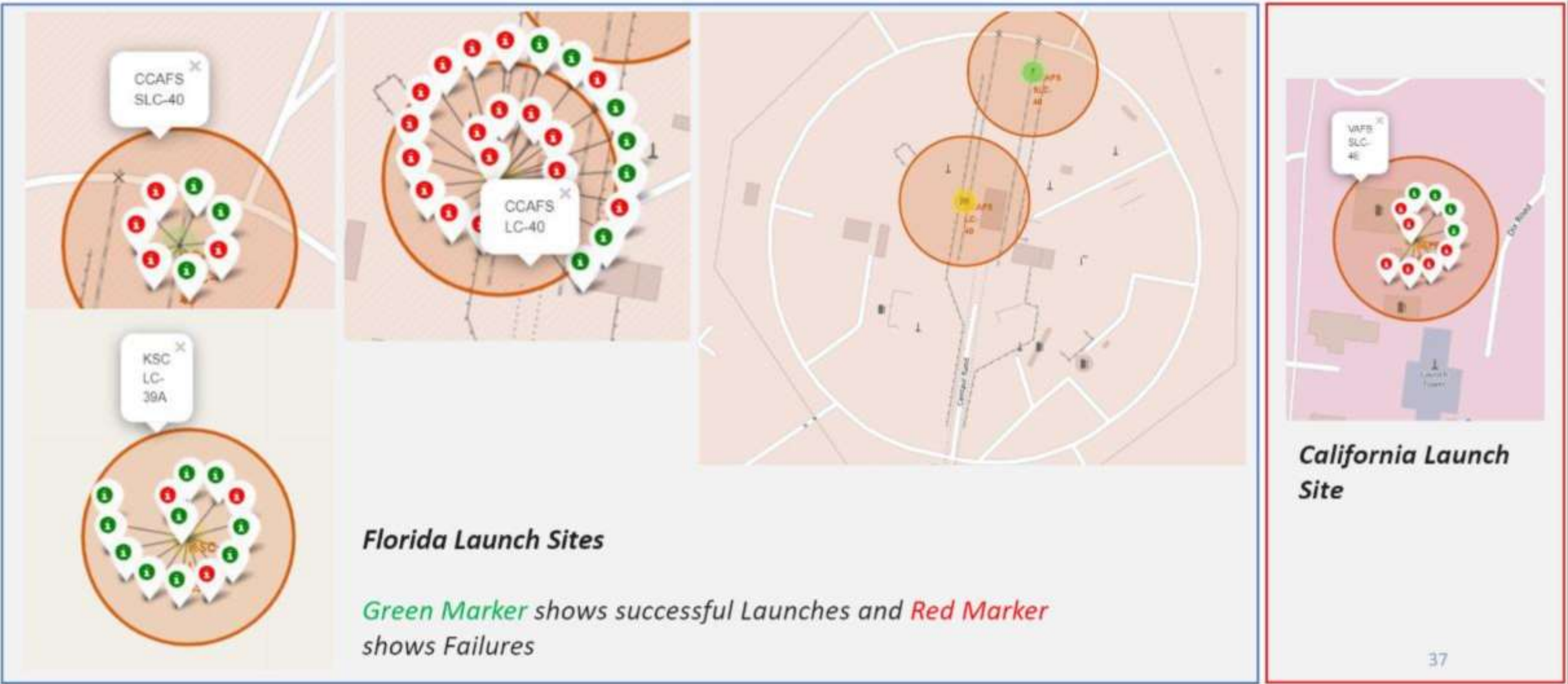
Global Launch Site Map

All four launch sites sit on the U.S. coastline, in Florida and California.



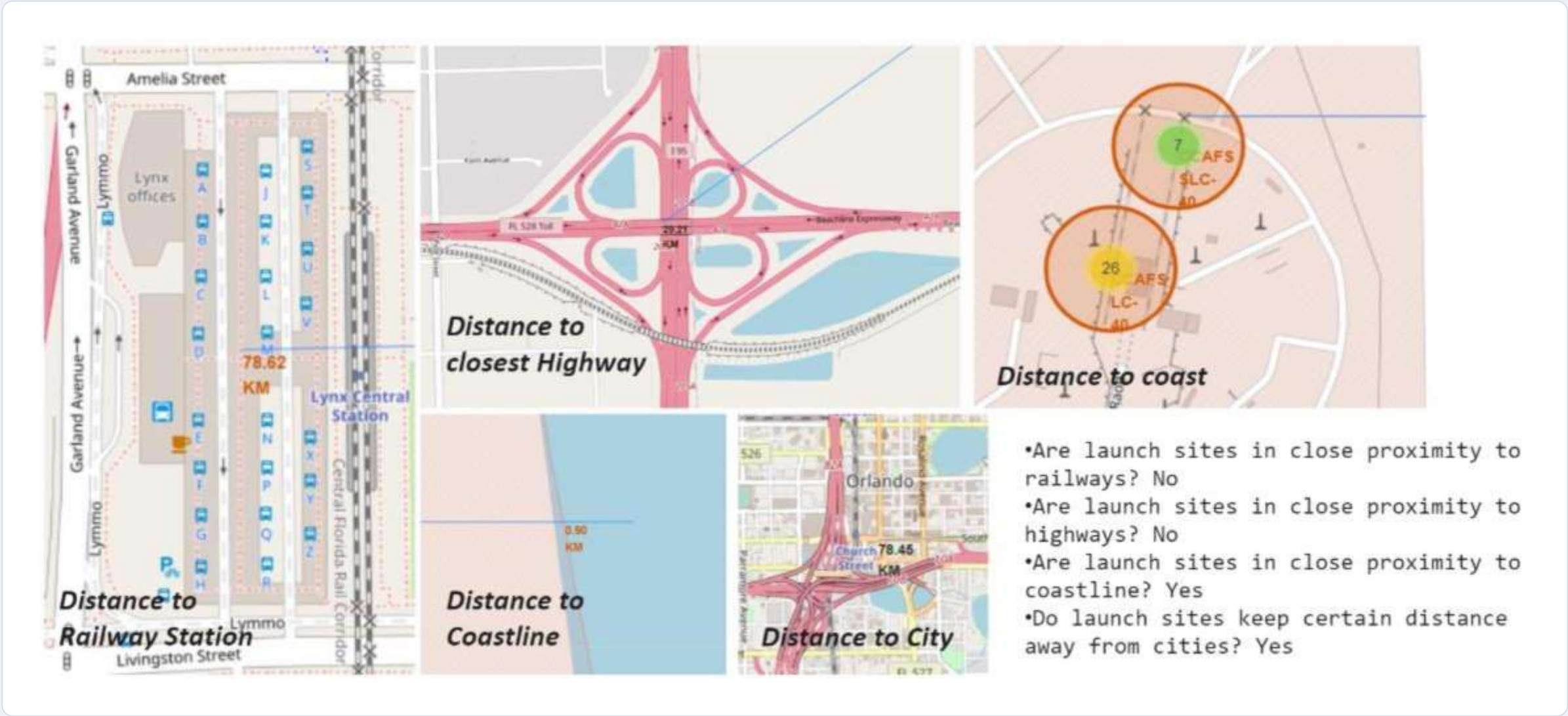
Launch Outcomes by Marker Color

Color-coded clusters — green for success, red for failure — highlight which sites land most reliably.



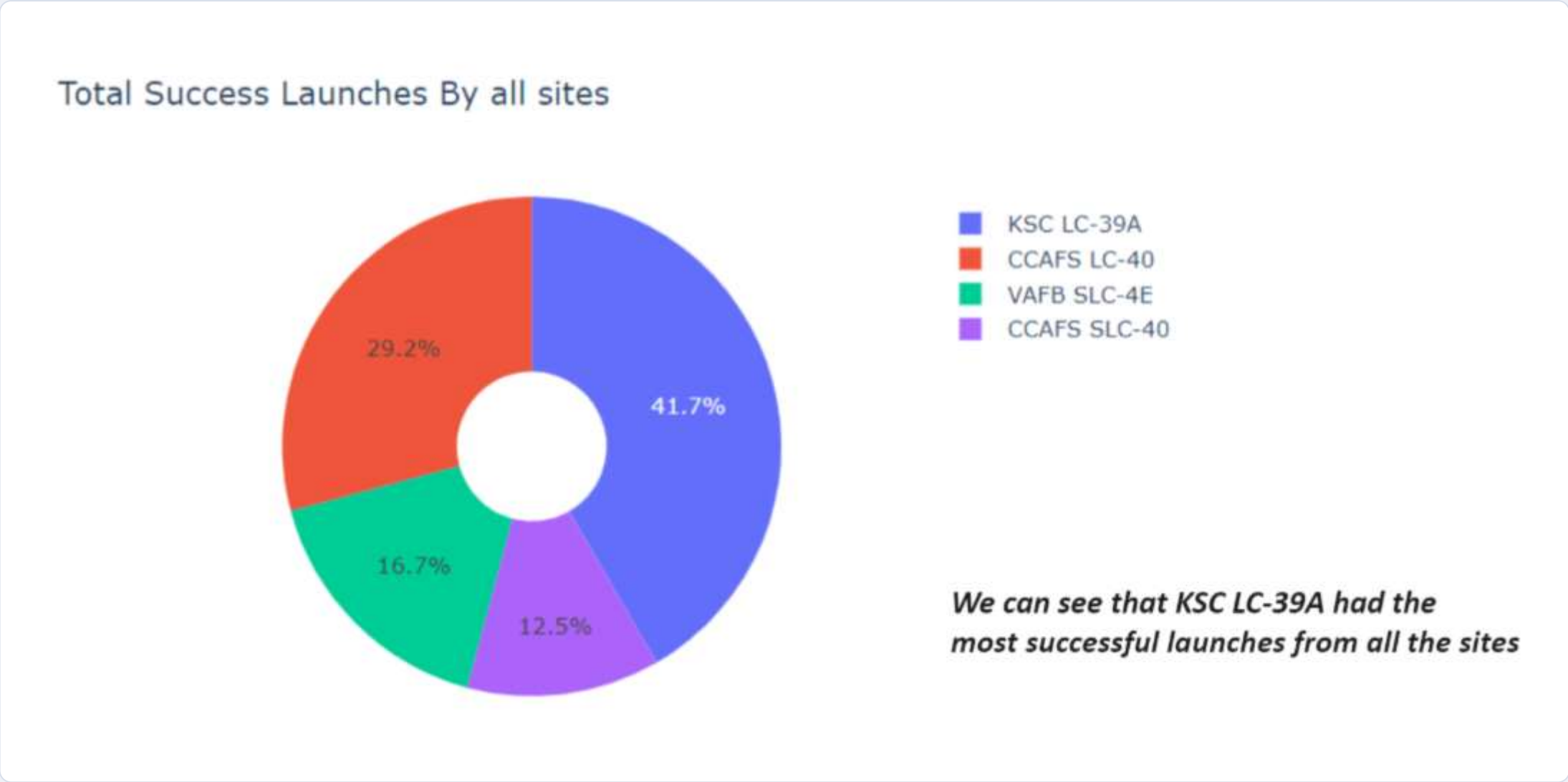
Launch Site Proximity Analysis

Sites sit close to highways and coastlines, but keep a deliberate distance from nearby cities.



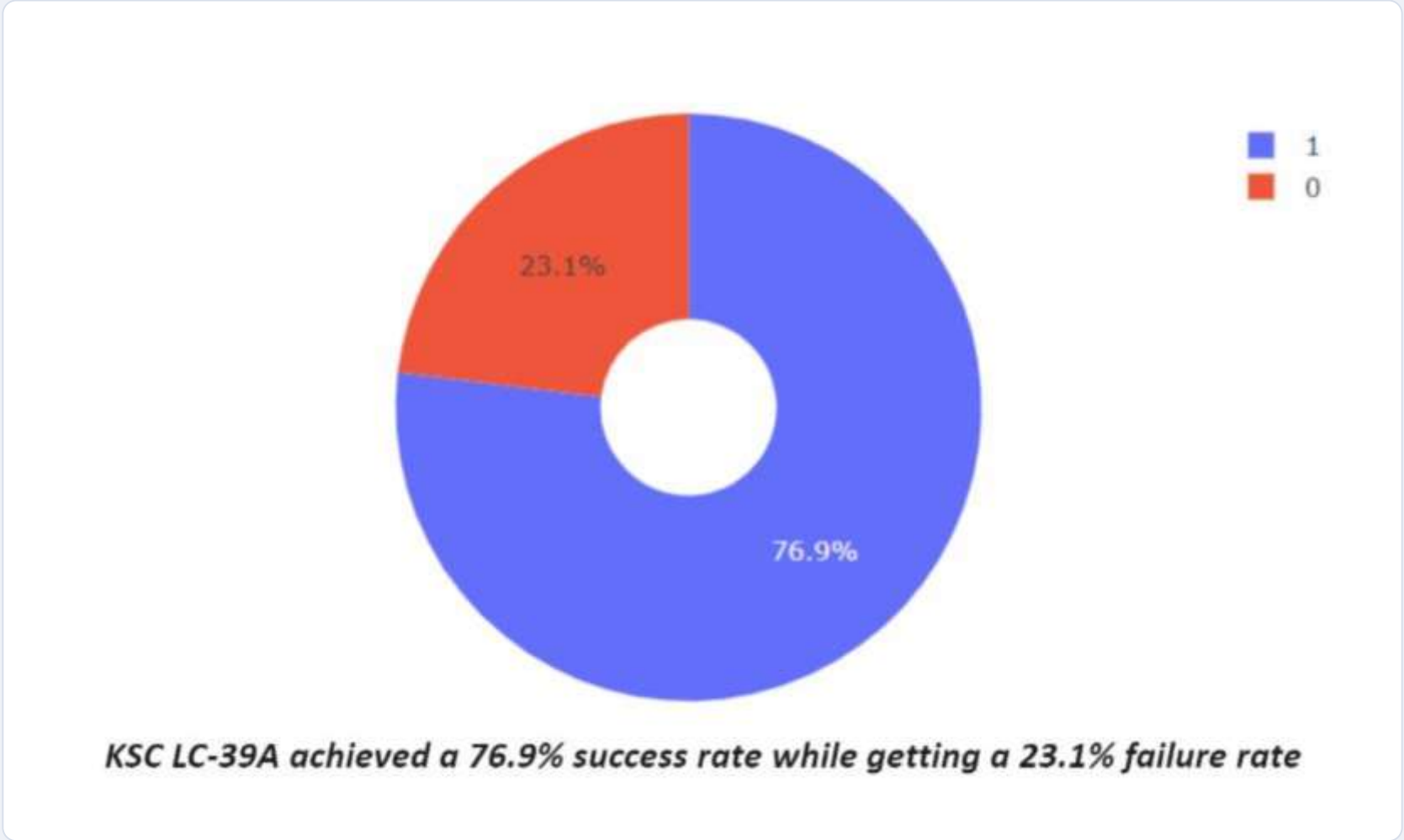
Success Share by Site

CCAFS SLC-40 and KSC LC-39A account for most launches — and most successes.



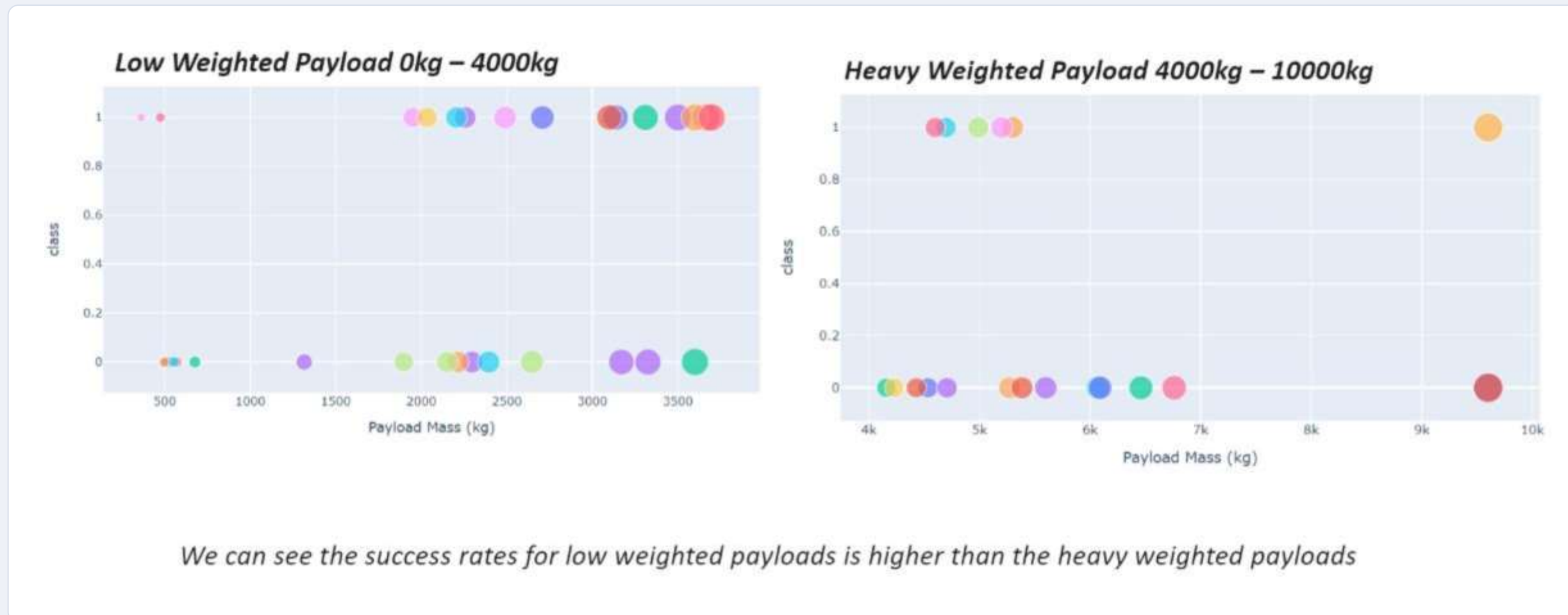
Highest Success-Ratio Site

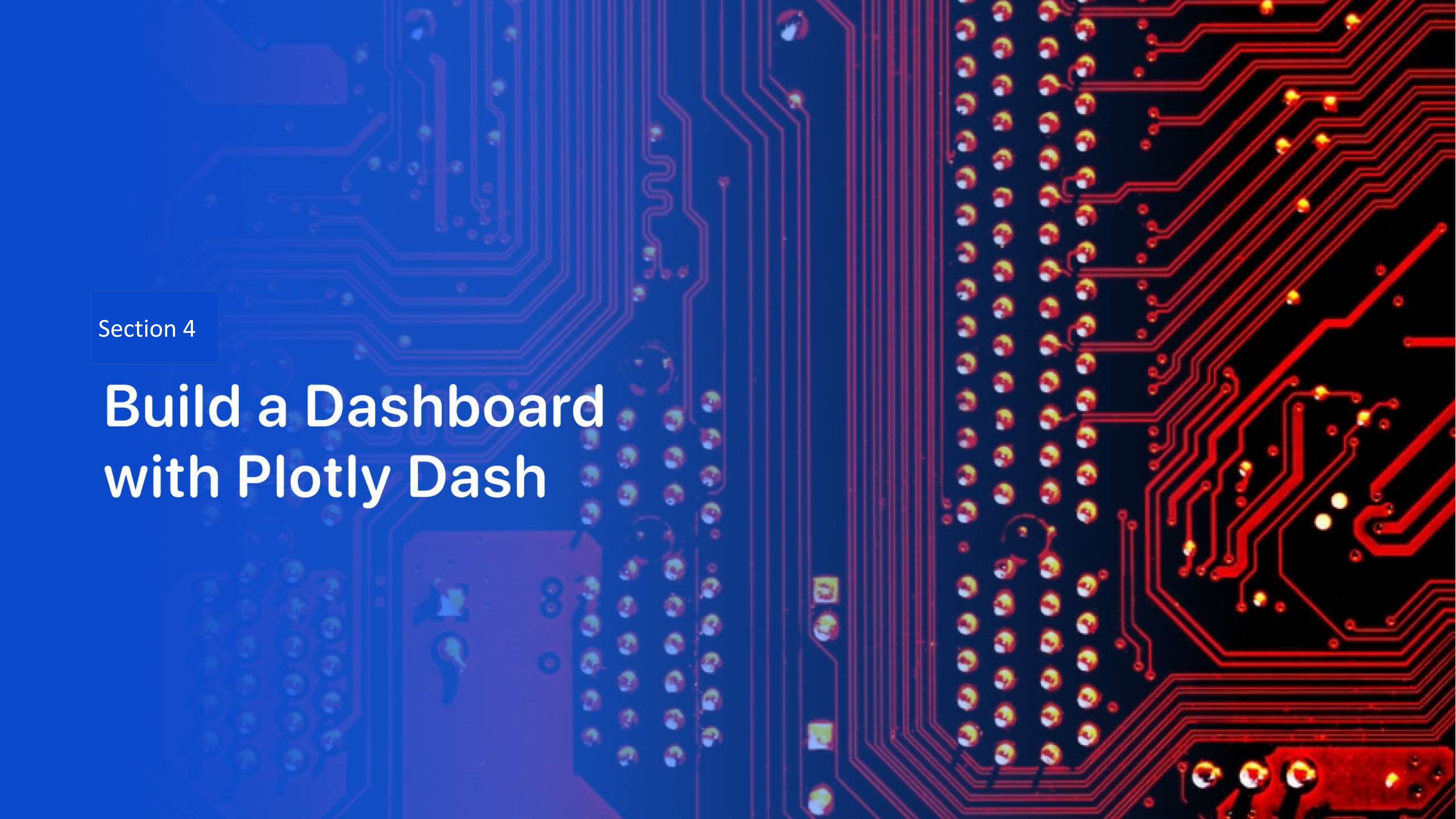
KSC LC-39A achieves a **76.9%** success rate against a 23.1% failure rate.



Payload vs. Outcome by Range

Success rates are higher for lighter payloads than for the heaviest bracket, filtered live by the range slider.





Section 4

Build a Dashboard with Plotly Dash

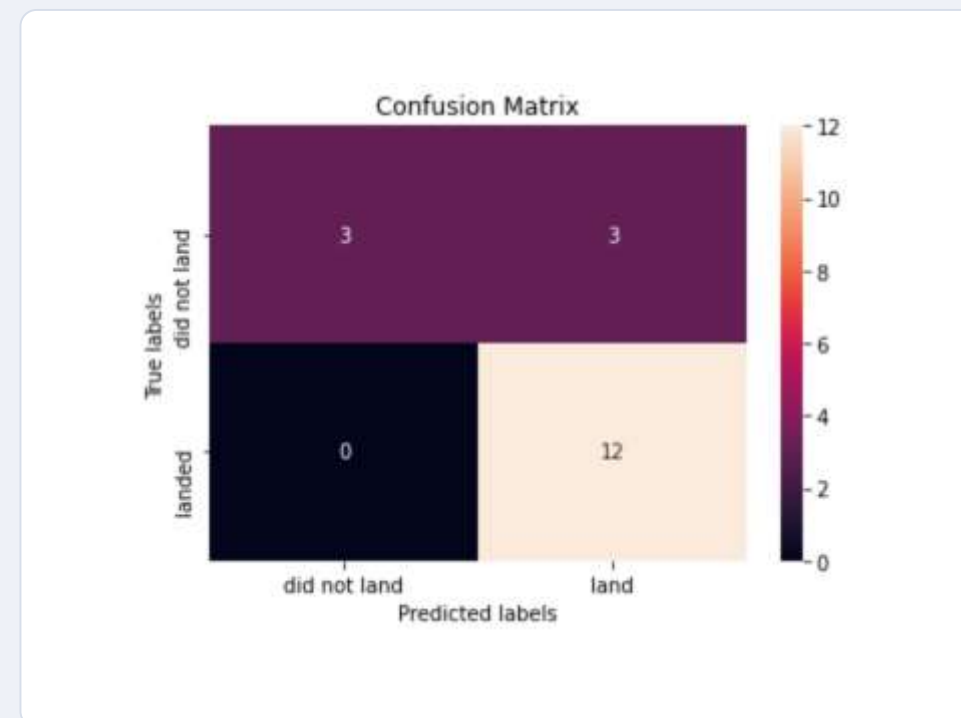
Classification Model Comparison

Four models were tuned with GridSearchCV and compared on classification accuracy.

Logistic Regression	Tuned with GridSearchCV
Support Vector Machine	Tuned with GridSearchCV
K-Nearest Neighbors	Tuned with GridSearchCV
Decision Tree	Best performing

Confusion Matrix

The main error is false positives — unsuccessful landings the model marks as successful.



CONCLUSION

Conclusions

- 01 Success rate improves as a launch site accumulates more flights
- 02 Launch success climbed steadily from 2013 through 2020
- 03 ES-L1, GEO, HEO, SSO and VLEO orbits show the strongest success rates
- 04 KSC LC-39A recorded the most successful launches of any site
- 05 The decision tree classifier is the best-performing model for this task

Thank you!

